Strings in C

A deep guide — memory, copying, and the stuff that bites

One topic, in depth, weighted toward memory. Starts from the fact that C has **no string type** (the `std::string` contrast), then where a string's memory lives (stack, literal, heap), length vs capacity and the null terminator, copying (`strcpy` , `strncpy` , `memcpy` , `strdup`), concatenation and growing buffers with `realloc` , safe formatting with `snprintf` , comparing and searching, and the security angle (buffer overflows, over-reads) that makes bounded copies matter. This is the companion to the File I/O guide.

1. First: there is no string type in C

You sensed this correctly. C has **no built-in string type**. Where C++ hands you `std::string` — an object that stores its own length, grows itself, bounds-checks, and frees its own memory — C gives you only a *convention*: a **string is an array of `char` ending in a `'\0'` byte**. That's the whole definition. `<string.h>` isn't a string class; it's a bag of plain functions that operate on those char arrays.

Everything `std::string` does for you, you do by hand in C:

	C++ `std::string`	C "string" (`char[]` + `'\0'`)
What it is	an object (a class)	a convention over a char array
Knows its length	yes, `.size()` is O(1)	no — `strlen` walks to `'\0'`, O(n)
Grows automatically	yes	no — you re-allocate yourself
Bounds-checked	yes (`.at()`)	no — overflow is on you
Owns / frees memory	yes (RAII)	no — you `malloc` / `free` it
Operations	methods (`.substr` , `+`)	free functions (`strcpy` , `strcat`)

So this guide leans on memory and copying on purpose — that's where all the real work, and all the bugs, actually live.

2. Where a string's memory lives

This is the single most important thing to get straight: it decides whether you can modify the string, how long it lives, and who frees it. There are three homes.

Stack array — you own it, fixed size, automatic lifetime.

```
char s[32] = "hello";   // 32 bytes on the stack, mutable
s[0] = 'H';             // fine
```

Lives until the function returns. Mutable. Size fixed at compile time — write more than 31 chars + `'\0'` and you overflow.

String literal — read-only, lives forever, never modify.

```
char *p = "hello";      // p points to read-only static memory
p[0] = 'H';             // UNDEFINED BEHAVIOR -- usually a crash
```

The text `"hello"` sits in a read-only section of the program; `p` just points at it. Read it, never write it.

Heap — you size it at runtime, you free it.

```
char *s = malloc(32);   // 32 bytes on the heap
strcpy(s, "hello");     // fill it
/* ... */
free(s);                // YOU must free it, exactly once
```

Mutable, lives until you `free` it, size chosen at runtime. This is the C equivalent of what `std::string` does internally — except you hold the controls.

Home	Mutable?	Lifetime	Who frees
Stack array	yes	until the function returns	automatic
String literal	**no**	whole program	nobody (static)
Heap (`malloc`)	yes	until you `free` it	**you**

The classic trap — returning a pointer to a stack array:

```c
char *make_name(void) {
    char buf[32];
    strcpy(buf, "Praveen");
    return buf;          // BUG: buf dies when the function returns
}
```

The returned pointer dangles — `buf` is gone. Fix it by allocating on the heap (`malloc` / `strdup`) and returning that, or by having the caller pass a buffer in.

3. Length, capacity, and the null terminator

Because a C string carries no length field, two different sizes matter and *you* track them:

- **Length** — characters before the `'\0'`. `strlen(s)` computes it by *walking the array* to the `'\0'`. That's O(n) every call — don't put it in a loop condition over the same string.
- **Capacity** — how many bytes the storage actually holds (`32` for `char s[32]`, `n` for `malloc(n)`). The string library never knows this; only you do.

```c
char s[32] = "cat";
strlen(s);    // 3  -- length (walks to '\0')
sizeof(s);    // 32 -- capacity (works on a real array, not a pointer)
```

The golden rule falls out of this: **a string of N visible characters needs N+1 bytes.** Forgetting the `+1` for `'\0'` is the root of a huge share of C bugs.

4. Copying strings — the core skill

There is no `=` for strings in C (`a = b` on arrays doesn't copy; on pointers it just aliases). Copying is always a function call, and picking the right one is most of the game.

`strcpy` — simple, and dangerous.

```c
char *strcpy(char *dst, const char *src);   // copies through '\0'
```

Copies `src` into `dst` including the terminator. It has **no idea how big `dst` is** — if `src` doesn't fit, it overflows and corrupts memory (the classic buffer overflow). Safe only when you're certain `dst` is big enough.

`strncpy` — bounded, but a footgun.

```c
char *strncpy(char *dst, const char *src, size_t n);
```

Copies at most `n` bytes, with two surprises:

- If `src` is **longer than `n`**, it copies `n` bytes and writes **no `'\0'`** — an unterminated string.
- If `src` is **shorter than `n`**, it pads the rest of `n` with `'\0'`.

The only correct way to use it on a fixed buffer is to force termination yourself:

```c
char dst[16];
strncpy(dst, src, sizeof(dst) - 1);
dst[sizeof(dst) - 1] = '\0';          // guarantee the terminator
```

`memcpy` — raw bytes, when you already know the length.

```c
void *memcpy(void *dst, const void *src, size_t n);
```

Copies exactly `n` bytes, ignores `'\0'`, doesn't scan — faster when you know the length (from `strlen` or a protocol field). You handle the terminator. Source and destination **must not overlap**.

`memmove` — the same, but overlap-safe. Use it when the ranges might overlap (shifting characters inside one buffer).

`strdup` — allocate and copy in one step.

```c
char *strdup(const char *s);   // POSIX; standard since C23
```

`malloc`s exactly enough, copies the string in, returns the new buffer — the clean way to put a copy on the heap. **You own the result and must `free` it.** Under the hood it's just:

```c
char *my_strdup(const char *s) {
    size_t n = strlen(s) + 1;      // +1 for '\0'
    char *copy = malloc(n);
    if (copy) memcpy(copy, s, n);  // copy includes the '\0'
    return copy;
}
```

Which one when:

Situation	Use
dst is definitely big enough	`strcpy` (or better, `snprintf`)
fixed buffer, might not fit	`snprintf`, or `strncpy` + manual `'\0'`
you already know the length	`memcpy`
the ranges overlap	`memmove`
you want a heap copy	`strdup`

5. Concatenation and building strings

`strcat` / `strncat` — append.

```c
char *strcat(char *dst, const char *src);
char *strncat(char *dst, const char *src, size_t n);
```

`strcat` appends `src` to the end of `dst`; `dst` must already hold enough spare room for `src` + `'\0'`, or you overflow. `strncat` appends at most `n` bytes of `src` then **always** adds `'\0'` (nicer than `strncpy` that way) — but its `n` is "max bytes to take from `src`", **not** the size of `dst`. Passing `sizeof(dst)` is a very common mistake.

Building dynamically with `malloc` / `realloc`. When you don't know the final size, grow the buffer — the same capacity-doubling idea as your dynamic array, but for `char`:

```c
// append src onto heap string *dst (length *len, capacity *cap)
void append(char **dst, size_t *len, size_t *cap, const char *src) {
    size_t add = strlen(src);
    if (*len + add + 1 > *cap) {
        *cap = (*len + add + 1) * 2;
        *dst = realloc(*dst, *cap);
    }
    memcpy(*dst + *len, src, add + 1);   // copy incl '\0'
    *len += add;
}
```

That's essentially a hand-rolled `std::string::operator+=`.

6. Formatting into a buffer — the safe workhorse

```c
int snprintf(char *buf, size_t size, const char *fmt, ...);
```

`snprintf` writes at most `size-1` characters plus `'\0'`, so it **can't overflow**, and it returns the length it *would* have written — a return `>= size` means truncation. It's the safest way to copy-and-join text:

```c
char path[64];
int n = snprintf(path, sizeof(path), "%s/%s.txt", dir, name);
if (n >= (int)sizeof(path)) {
    /* truncated -- buffer too small */
}
```

Prefer it over `strcpy` + `strcat` chains: one call, bounds-checked, always terminated. Never use its unsafe sibling `sprintf`, which has no size limit.

7. Comparing and searching

Compare:

```c
int strcmp(const char *a, const char *b);
int strncmp(const char *a, const char *b, size_t n);
```

Returns `0` if equal, `<0` / `>0` for ordering — **not** a boolean, since equal is `0`:

```
if (strcmp(a, b) == 0) { /* match */ }
```

The nastiest beginner bug: `if (a == b)` compares the two **pointers** (addresses), not the text. Always `strcmp`.

Search (each returns a pointer into the string, or `NULL`):

```
char *strchr (const char *s, int c);           // first c
char *strrchr(const char *s, int c);           // last c
char *strstr (const char *s, const char *sub);  // substring
size_t strspn (const char *s, const char *set); // prefix length in set
size_t strcspn(const char *s, const char *set); // prefix length NOT in set
```

`strcspn` is how you strip a trailing newline from `fgets`: `s[strcspn(s, "\n")] = '\0';`. And `memcmp(a, b, n)` compares `n` raw bytes when you don't want to stop at `'\0'`.

8. The security angle (why copying safely matters)

Because you do your own memory management, strings are where C programs get *exploited* — which is exactly why bounded copies matter, and worth knowing for CTF:

- **Buffer overflow.** `strcpy` / `strcat` / `sprintf` / `gets` writing past a stack buffer can overwrite the return address — the classic stack smash. Mitigation: bounded functions with explicit sizes (`snprintf`, `strncat`), and never `gets`.
- **Missing terminator → over-read.** An unterminated string (a careless `strncpy`) makes `strlen` / `printf("%s")` run off the end, reading adjacent memory — an information leak. Always ensure a `'\0'`.
- **Off-by-one.** Forgetting the `+1` for `'\0'` writes one byte past the buffer — small, but still corruption.

Where available, the BSD helpers `strlcpy` / `strlcat` beat `strncpy` / `strncat`: they always null-terminate and take the real buffer size. They aren't in ISO C (and only recently in glibc), so they're not everywhere — but `snprintf` is, and it covers most cases.

9. Worked examples

Safe copy into a fixed buffer:

```
char name[32];
snprintf(name, sizeof(name), "%s", user_input);   // can't overflow
```

Duplicate a string onto the heap (and free it):

```
char *copy = strdup(original);   // malloc + copy
if (!copy) { /* out of memory */ }
/* ... use copy ... */
free(copy);
```

Join an array of strings into one heap buffer:

```
char *join(char **parts, int count) {
    size_t total = 1;                  // room for '\0'
    for (int i = 0; i < count; i++)
        total += strlen(parts[i]);

    char *out = malloc(total);
    if (!out) return NULL;
    out[0] = '\0';
    for (int i = 0; i < count; i++)
        strcat(out, parts[i]);         // safe: total was pre-sized
    return out;                        // caller frees
}
```

10. Best-practices checklist

- Remember there's no string type — you manage the memory, always.
- Size every buffer for the worst case; a string of N chars needs N+1 bytes.
- Prefer `snprintf` for copying/joining into fixed buffers; avoid `strcpy` / `sprintf` / `gets`.
- After `strncpy`, force a `'\0'` yourself.
- `strncat`'s count is bytes-from-src, not the buffer size.
- Never compare strings with `==`; use `strcmp(...) == 0`.
- Don't return a pointer to a local (stack) char array — heap-allocate or take a caller buffer.

- Every `malloc` / `strdup` / `realloc` string gets exactly one `free`.
- Walk characters with `<ctype.h>` (`toupper`, `isdigit`, …) and an `(unsigned char)` cast.

References & further reading

- **cppreference — null-terminated byte strings:** https://en.cppreference.com/w/c/string/byte
- **Linux man pages (section 3):** `man 3 strcpy`, `man 3 strncpy`, `man 3 strdup`, `man 3 strcat`, `man 3 snprintf` — or https://man7.org/linux/man-pages/
- **Secure coding:** SEI CERT C Coding Standard, the STR (string) rules — https://wiki.sei.cmu.edu/confluence/display/c
- **K&R, *The C Programming Language* (2nd ed.):** Chapter 5 (Pointers and Arrays) and the `<string.h>` summary in Appendix B3.